# Structure Complète du Backend Blueprint

Le document d’architecture backend doit contenir les sections suivantes :

1. Executive Summary
2. High-Level Architecture
3. Business Domains & Bounded Contexts
4. Detailed Use Cases
5. Functional Requirements Mapping
6. Domain Model
7. MCD (Modèle Conceptuel de Données)
8. UML Class Diagrams
9. Database Schema (PostgreSQL)
10. API Design
11. Authentication & Security
12. Authorization Model
13. Service Layer Architecture
14. Repository Layer
15. AI Orchestration Layer
16. Recommendation Engine
17. Pantry Recognition Pipeline
18. Budget Optimization Engine
19. Notification System
20. Subscription & Billing
21. Event-Driven Architecture
22. Caching Strategy
23. File Storage
24. Monitoring & Observability
25. Testing Strategy
26. CI/CD
27. Infrastructure on Amazon Web Services
28. Performance & Scalability
29. Data Privacy & Compliance
30. Development Roadmap

# 1. Executive Summary

## PlatePilot Backend Architecture Blueprint v1.0

**Project:** PlatePilot
**Backend Stack:** Spring Boot 3.x, Java 21, PostgreSQL 16, Redis 7, Amazon Web Services, OpenAI API, Google ML Kit
**Architecture Style:** Modular Monolith with Domain-Driven Design (DDD)
**Primary Database:** PostgreSQL
**Caching Layer:** Redis
**Deployment Target:** AWS App Runner or Amazon ECS
**Security:** Spring Security + JWT + OAuth2
**API Style:** RESTful JSON API
**Target Clients:** Flutter mobile application (iOS and Android)

---

## 1.1 Purpose of This Document

This document defines the architectural vision and technical foundation of the PlatePilot backend.

Its purpose is to provide a clear and authoritative blueprint for designing, implementing, deploying, and scaling the server-side platform that powers all core features of the product.

The backend is responsible for:

- User authentication and authorization
- Preference management
- Pantry inventory management
- Weekly meal plan generation
- Grocery list generation
- Budget management
- Recipe management
- Notifications
- Premium subscription management
- Analytics and event tracking
- AI orchestration and recommendation logic

This document serves as the technical reference for all backend development decisions.

## 1.2 Product Context

PlatePilot is an AI-powered meal planning and grocery assistant designed to reduce the cognitive burden of deciding what to cook and what to buy.

The application generates personalized meal plans and shopping lists based on:

- Household size
- Weekly food budget
- Available cooking time
- Cooking skill level
- Dietary restrictions
- Allergies
- Ingredients already available at home
- User goals such as saving money or reducing food waste

The core product promise is:

> **In less than one minute, PlatePilot creates a personalized weekly meal plan and an intelligent grocery list.**

The backend is the central system that processes all these constraints and produces optimized recommendations.

## 1.3 Architectural Objectives

The backend architecture has been designed to achieve the following objectives.

### Functional Objectives

- Support all MVP and post-MVP business capabilities
- Orchestrate recommendation and optimization workflows
- Expose secure APIs to the Flutter application
- Integrate external AI and automation services

### Quality Objectives

- High maintainability
- Clear separation of concerns
- Strong testability
- Security by design
- Operational observability

- Horizontal scalability
- Cost efficiency

### Business Objectives

- Enable rapid product iteration
- Support subscription monetization
- Minimize technical debt
- Provide a stable foundation for international expansion

## 1.4 Architecture Style

The recommended architecture is a **Modular Monolith** implemented with **Domain-Driven Design (DDD)** principles.

### Why Modular Monolith

For an early-stage product, a modular monolith offers significant advantages over microservices:

- Single deployable unit
- Lower operational complexity
- Strong module boundaries
- Easier debugging and testing
- Faster development velocity
- Reduced infrastructure costs

### Why Domain-Driven Design

DDD aligns the codebase with business concepts and creates clear boundaries between functional domains such as Pantry, Meal Planning, and Budget Management.

This leads to:

- Better maintainability
- Improved team collaboration
- Explicit domain rules
- Easier future extraction into microservices

# 1.5 Technology Stack

## Core Platform

- Java 21
- Spring Boot 3.x
- Maven

## Persistence

- PostgreSQL
- Spring Data JPA
- Flyway

## Security

- Spring Security
- JWT access and refresh tokens
- OAuth2 social login

## Performance

- Redis

## API

- REST over HTTPS
- OpenAPI / Swagger

## Observability

- Micrometer
- Prometheus
- Grafana
- Sentry

## Cloud Infrastructure

- Amazon Web Services
- AWS App Runner or Amazon ECS
- Amazon RDS for PostgreSQL
- Amazon ElastiCache for Redis
- Amazon S3

### AI Services

- OpenAI API Platform
- Google ML Kit (used primarily on the mobile side)

## 1.6 Core Business Capabilities Supported

The backend will support the following domains:

- Authentication and user management
- Preference management
- Budget management
- Pantry inventory
- Recipe catalog
- Weekly meal planning
- Grocery list generation
- Recommendation engine
- Notifications
- Premium subscriptions
- Analytics and event tracking

Each capability will be implemented as an independent module with well-defined boundaries.

## 1.7 Key Architectural Principles

1. **Separation of Concerns**
   Presentation, application, domain, and infrastructure responsibilities are isolated.
2. **Single Responsibility**
   Classes and modules focus on one responsibility.
3. **Explicit Domain Modeling**
   Business rules are represented directly in the domain model.
4. **Security by Design**
   Authentication, authorization, and validation are integral.
5. **Testability**
   All modules are designed for unit and integration testing.
6. **Observability First**
   Metrics, logs, and traces are treated as first-class concerns.
7. **API Contract Stability**
   Public APIs are versioned and documented.

8. **Cloud-Native Deployment**
   Infrastructure is containerized and automated.

## 1.8 Expected Scale

### MVP Scale

- Hundreds to thousands of users
- Tens of thousands of API calls per day

### Growth Stage

- Tens of thousands of active users
- High-frequency recommendation workloads

The modular monolith architecture is more than sufficient for these stages and can be evolved later if necessary.

## 1.9 Security Objectives

The backend must ensure:

- Secure authentication
- Role-based authorization
- Encrypted transport via HTTPS
- Secure password hashing
- Token rotation
- Rate limiting
- Auditability of sensitive actions

## 1.10 Strategic Conclusion

The PlatePilot backend architecture is designed as a robust and scalable foundation for a premium AI-driven meal planning platform.

By combining:

- Spring Boot
- PostgreSQL

- Redis
- Amazon Web Services
- OpenAI APIs
- Domain-Driven Design
- Modular Monolith architecture

we establish a platform that is:

- Technically sound
- Secure
- Maintainable
- Cost-efficient
- Operationally mature
- Ready for rapid product evolution

This architecture provides the technical backbone that will enable PlatePilot to deliver on its promise of reducing meal-planning stress and becoming a trusted weekly companion for users worldwide.

# 2. High-Level Architecture

## 2.1 Architectural Philosophy

L’architecture backend de PlatePilot suit une approche **Modular Monolith + Domain-Driven Design (DDD)**.

Cette architecture repose sur les principes suivants :

- Une seule application déployable.
- Des modules métier strictement séparés.
- Des dépendances maîtrisées entre modules.
- Une API REST unique.
- Une base de données relationnelle centralisée.
- Une couche d’orchestration IA.
- Une infrastructure cloud-native.

Cette approche combine la simplicité opérationnelle d’un monolithe et la modularité conceptuelle d’une architecture distribuée.

## 2.2 System Context

Le backend agit comme le cœur décisionnel de PlatePilot.

Il reçoit des données provenant du frontend Flutter Flutter, applique les règles métier, interagit avec les services IA et renvoie des recommandations personnalisées.

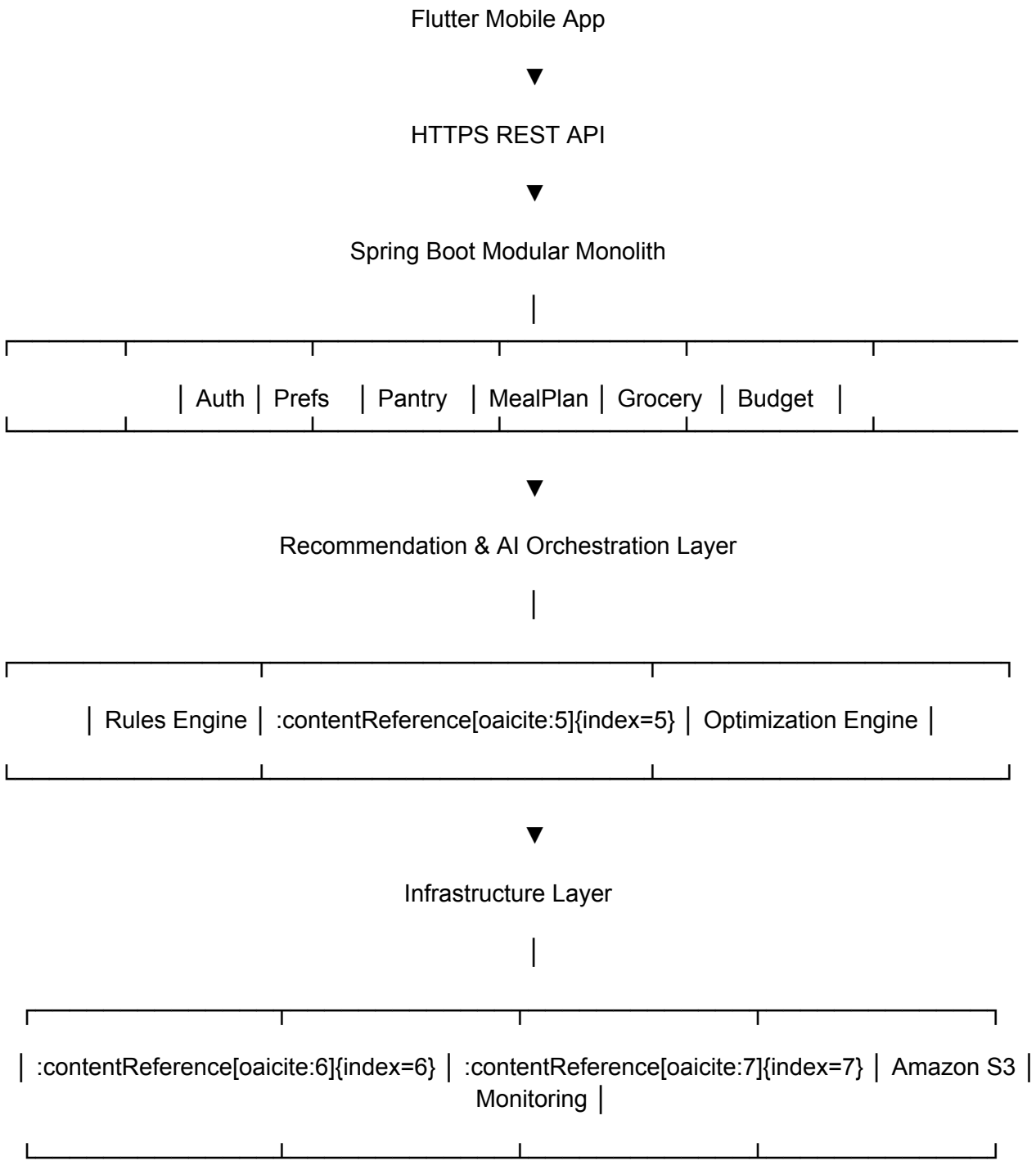
### Inputs principaux

- Données d'onboarding
- Préférences utilisateur
- Budget hebdomadaire
- Pantry inventory
- Historique d'utilisation
- Actions utilisateur

### Outputs principaux

- Plans de repas hebdomadaires
- Listes de courses
- Suggestions rapides
- Notifications
- Statistiques d'économies

## 2.3 High-Level System Diagram



## 2.4 Layered Architecture

L’application backend est organisée en couches clairement séparées.

### 1. Presentation Layer

Responsable de l’exposition des APIs REST.

Composants :

- REST Controllers
- Model
- Repositories
- Services
- COnfiguration
- Request DTOs
- Response DTOs
- Validation annotations
- Exception handlers

### 2. Application Layer

Orchestre les cas d’usage.

Composants :

- Use Cases
- Application Services
- Command/Query handlers
- Transaction boundaries

### 3. Domain Layer

Contient les règles métier et le modèle du domaine.

Composants :

- Aggregates
- Entities
- Value Objects
- Domain Services
- Domain Events

### 4. Infrastructure Layer

Implémente les détails techniques.

Composants :

- JPA repositories
- Redis adapters
- OpenAI adapters
- S3 storage
- Notification providers

### 5. Shared Kernel

Éléments transverses réutilisables.

Composants :

- Base entities
- Audit classes
- Common exceptions
- Utilities

## 2.5 Runtime Request Flow

### Example: Generate Weekly Meal Plan

1. Le frontend Flutter appelle `POST /api/v1/meal-plans/generate`.
2. Le `MealPlanController` valide la requête.
3. Le `GenerateMealPlanUseCase` est invoqué.
4. Les préférences, le budget, le pantry et les recettes sont chargés.
5. Le Recommendation Engine applique les contraintes métier.
6. L'AI Orchestration Layer peut enrichir les suggestions.
7. Le plan généré est persisté dans PostgreSQL.
8. Les résultats sont mis en cache dans Redis.
9. Une réponse JSON structurée est renvoyée au mobile.

## 2.6 Deployment Architecture

### Containerization

L'application est empaquetée dans une image Docker.

### Cloud Deployment

Déploiement recommandé sur Amazon Web Services via :

- AWS App Runner (MVP)
- Amazon ECS (croissance)

### Managed Services

- Amazon RDS for PostgreSQL
- Amazon ElastiCache for Redis
- Amazon S3
- Amazon CloudWatch

## 2.7 Integration Architecture

### Mobile Client

Flutter communicates with the backend over HTTPS using JSON APIs.

### AI Services

- OpenAI for contextual meal reasoning.
- Google ML Kit is primarily used on-device by the mobile app.

### External Services

- Payment provider (future integration with Stripe)
- Push notifications (e.g. Google Firebase Cloud Messaging, Spring Messages)

## 2.8 Data Flow Architecture

### Primary Data Flow

1. User submits preferences and pantry data.
2. Backend validates and stores the information.
3. Recommendation Engine computes optimal meals.
4. Grocery Generator derives shopping requirements.
5. Results are persisted and cached.
6. Analytics events are recorded.
7. Notifications may be generated.

### Secondary Data Flow

- Budget updates adjust recommendation constraints.
- Pantry changes trigger recalculation suggestions.
- Subscription changes unlock premium capabilities.

## 2.9 Architectural Decisions Summary

| Decision | Rationale |
|---|---|
| Modular Monolith | Best balance between simplicity and modularity |
| Domain-Driven Design | Aligns code with business concepts |
| REST API | Native fit for Flutter mobile clients |
| PostgreSQL | Strong relational integrity and SQL capabilities |
| Redis | Fast cache and ephemeral state |
| Flyway | Version-controlled database migrations |
| JWT + Refresh Tokens | Secure stateless authentication |
| AWS App Runner | Simplified initial deployment |
| OpenAI API | Advanced recommendation enrichment |

## 2.10 High-Level Architecture Conclusion

The high-level architecture of PlatePilot is designed to provide a stable and scalable foundation for all product capabilities.

It separates concerns across:

- Mobile presentation
- Secure API exposure
- Application orchestration
- Domain logic
- AI orchestration
- Persistent storage
- Cloud infrastructure

This architecture ensures that PlatePilot can evolve rapidly while maintaining strong modularity, security, and operational reliability.

---

# 3. Business Domains & Bounded Contexts

## 3.1 Domain-Driven Design Strategy

The backend is organized according to Domain-Driven Design (DDD).

Each major business capability is implemented as a **Bounded Context** with:

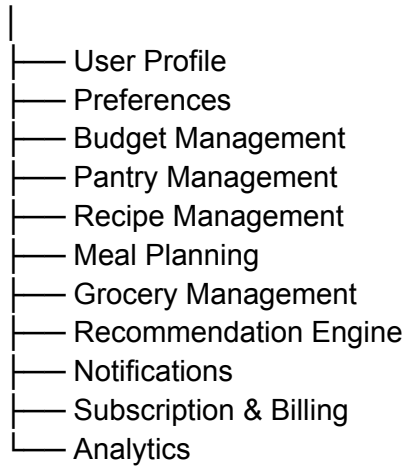
- Its own domain model
- Application services
- Repositories
- REST endpoints
- Internal business rules

This approach minimizes coupling and clarifies ownership of responsibilities.

---

## 3.2 Domain Map Overview

Identity & Access



# 3.3 Core Bounded Contexts

## 1. Identity & Access Context

Responsible for authentication, authorization, and account lifecycle.

**Responsibilities**

- User registration
- Login and logout
- Password reset
- Email verification
- Social authentication
- JWT and refresh token management
- Role and permission assignment
- Session revocation

**Main Aggregates**

- User
- Role
- Permission
- RefreshToken
- EmailVerificationToken
- PasswordResetToken

**Exposed APIs**

- `/api/v1/auth/register`
- `/api/v1/auth/login`
- `/api/v1/auth/refresh`
- `/api/v1/auth/logout`
- `/api/v1/auth/forgot-password`
- `/api/v1/auth/reset-password`

## 2. User Profile Context

Stores personal and account-related information.

**Responsibilities**

- Profile management

- Avatar handling
- Language selection
- Theme preferences
- Notification preferences

**Main Aggregates**

- UserProfile
- NotificationSettings

## 3. Preferences Context

Stores all personalization constraints used by the recommendation engine.

**Responsibilities**

- Household size
- Cooking skill
- Dietary restrictions
- Allergies
- Cooking time limits
- Preferred cuisines
- Goals

**Main Aggregates**

- UserPreferences
- DietaryRestriction
- Allergy
- UserGoal

## 4. Budget Management Context

Tracks weekly budgets and spending constraints.

**Responsibilities**

- Set weekly budget
- Add budget adjustments
- Replace budget
- Calculate remaining budget
- Budget history

**Main Aggregates**

- Budget
- BudgetAdjustment
- BudgetSnapshot

## 5. Pantry Management Context

Manages ingredients available at home.

**Responsibilities**

- Add/update/delete pantry items
- Expiration tracking
- Low-stock alerts
- Waste reduction suggestions
- Scan result integration

**Main Aggregates**

- PantryItem
- PantryCategory
- ExpirationAlert

## 6. Recipe Management Context

Maintains recipes and ingredient requirements.

**Responsibilities**

- Internal recipe catalog
- Custom recipes
- Ingredients
- Instructions
- Nutritional metadata

**Main Aggregates**

- Recipe
- RecipeIngredient
- RecipeStep
- NutritionInfo

## 7. Meal Planning Context

Stores weekly plans and meal selections.

**Responsibilities**

- Generate plans
- Replace meals
- Lock meals
- Track selected recipes

**Main Aggregates**

- MealPlan
- MealPlanDay
- MealSlot

## 8. Grocery Management Context

Generates and manages shopping lists.

**Responsibilities**

- Build lists from meal plans
- Group by category
- Exclude pantry items
- Check completion state
- Share list

**Main Aggregates**

- GroceryList
- GroceryItem

## 9. Recommendation Engine Context

Core decision engine that selects meals.

**Responsibilities**

- Constraint filtering
- Cost estimation
- Pantry utilization
- Meal ranking

- Variety optimization

**Main Components**

- ConstraintEvaluator
- CandidateGenerator
- RankingEngine
- OptimizationEngine

## 10. Notifications Context

Generates and manages user notifications.

**Responsibilities**

- Pantry alerts
- Budget alerts
- Weekly reminders
- Premium prompts

**Main Aggregates**

- Notification
- NotificationPreference

## 11. Subscription & Billing Context

Handles premium plans and payment state.

**Responsibilities**

- Subscription lifecycle
- Entitlements
- Billing status
- Feature access

**Main Aggregates**

- Subscription
- Plan
- FeatureEntitlement

### 12. Analytics Context

Captures product usage and business metrics.

**Responsibilities**

- Event tracking
- Funnel analysis
- Retention metrics
- Recommendation performance

**Main Aggregates**

- AnalyticsEvent
- UserMetric
- RecommendationMetric

## 3.4 Context Relationships

### Upstream Dependencies

- Meal Planning depends on Preferences, Budget, Pantry, and Recipes.
- Grocery Management depends on Meal Planning and Pantry.
- Recommendation Engine depends on Preferences, Budget, Pantry, and Recipes.
- Notifications depends on all major contexts.
- Subscription influences feature availability across the platform.

### Dependency Principle

Dependencies are unidirectional and controlled through interfaces and application services.

## 3.5 Shared Kernel

The following cross-cutting concepts are shared:

- BaseEntity
- AuditableEntity

- Money
- Quantity
- Unit
- DomainException
- DomainEvent

Only generic abstractions are shared to avoid tight coupling.

## 3.6 Context Prioritization for MVP

### Essential MVP Contexts

1. Identity & Access
2. Preferences
3. Budget Management
4. Pantry Management
5. Recipe Management
6. Meal Planning
7. Grocery Management
8. Recommendation Engine

### Near-Term Contexts

9. Notifications
10. Analytics

### Post-MVP Contexts

11. Subscription & Billing

## 3.7 Modular Package Structure

```
com.platepilot.modules
├── auth
├── user
├── preferences
├── budget
├── pantry
├── recipes
├── mealplan
├── grocery
├── recommendation
├── notifications
├── subscription
└── analytics
```

Each module contains:

- `domain/`
- `application/`
- `infrastructure/`
- `api/`

## 3.8 Strategic Conclusion

The bounded-context model ensures that every major business capability in PlatePilot is implemented as an isolated, cohesive module with explicit responsibilities and dependencies.

This structure provides:

- Strong modularity
- Clear ownership of business rules
- Reduced coupling
- Improved testability
- Easier future evolution

> By organizing the backend around business domains rather than technical layers alone, PlatePilot gains an architecture that mirrors the product itself and supports long-term maintainability and scalability.

# 4. Detailed Use Cases

# 4.1 Purpose

This section describes the core business interactions supported by the backend. Each use case corresponds to one or more application services and API endpoints.

# 4.2 Primary Actors

- End User
- Mobile Application (Flutter)
- Recommendation Engine
- AI Orchestration Service
- Payment Provider (future integration with Stripe)
- Push Notification Provider

# 4.3 Core Use Cases

## UC-01 Register Account

**Actor:** End User
**Goal:** Create a new account and initialize a profile.

### Preconditions

- Email is not already registered.

### Main Flow

1. User submits registration information.
2. Backend validates data.
3. Password is hashed.
4. User account is created.
5. Default profile and preferences are initialized.
6. Verification token is generated.
7. Tokens are returned.

### Postconditions

- User account exists and is authenticated.

## UC-02 Complete Onboarding

**Actor:** End User
**Goal:** Save preferences and constraints.

**Data Captured**

- Household size
- Weekly budget
- Cooking time
- Cooking skill
- Dietary restrictions
- Allergies
- Goals

**Postconditions**

- Preferences are stored and available to the recommendation engine.

---

## UC-03 Manage Pantry

**Actor:** End User
**Goal:** Add, update, and remove pantry items.

**Data**

- Ingredient
- Quantity
- Unit
- Expiration date

**Postconditions**

- Pantry inventory reflects the current home stock.

---

## UC-04 Generate Weekly Meal Plan

**Actor:** End User
**Goal:** Produce a seven-day plan optimized for user constraints.

**Inputs**

- Preferences
- Budget

- Pantry inventory
- Recipe catalog

**Outputs**

- Seven-day meal plan
- Estimated cost
- Prep times

---

## UC-05 Generate Grocery List

**Actor:** System
**Goal:** Derive required shopping items from the selected meal plan.

**Outputs**

- Grouped grocery list
- Quantities
- Pantry-aware exclusions

---

## UC-06 Quick Meal Suggestion

**Actor:** End User
**Goal:** Receive one or more fast meal suggestions.

**Inputs**

- Maximum preparation time
- Available ingredients
- Budget constraints

---

## UC-07 Manage Budget

**Actor:** End User
**Goal:** Set, replace, or adjust weekly budget.

**Outputs**

- Updated remaining budget
- Budget history

---

### UC-08 Edit Preferences

**Actor:** End User
**Goal:** Modify onboarding information after registration.

---

### UC-09 Receive Notifications

**Actor:** System
**Goal:** Inform the user about important events.

**Examples**

- Expiring ingredients
- Budget threshold alerts
- Weekly reminders

---

### UC-10 Upgrade to Premium

**Actor:** End User
**Goal:** Unlock premium capabilities.

---

# 5. Functional Requirements Mapping

| ID | Requirement | Primary Module |
|---|---|---|

| | | |
|---|---|---|
| FR-01 | User registration and login | Identity & Access |
| FR-02 | Save onboarding preferences | Preferences |
| FR-03 | Set and update budget | Budget |
| FR-04 | Manage pantry items | Pantry |
| FR-05 | Maintain recipe catalog | Recipes |
| FR-06 | Generate meal plans | Meal Planning |
| FR-07 | Generate grocery lists | Grocery |
| FR-08 | Produce quick meal suggestions | Recommendation |
| FR-09 | Send notifications | Notifications |
| FR-10 | Support subscriptions | Subscription |
| FR-11 | Record analytics events | Analytics |
| FR-12 | Support EN/FR localization settings | User Profile |

---

# 6. Domain Model

# 6.1 Core Aggregates

## User Aggregate

- User
- UserProfile
- NotificationSettings

## Preferences Aggregate

- UserPreferences
- DietaryRestriction
- Allergy
- UserGoal

## Budget Aggregate

- Budget
- BudgetAdjustment

## Pantry Aggregate

- PantryItem

## Recipe Aggregate

- Recipe
- RecipeIngredient
- RecipeStep

## Meal Planning Aggregate

- MealPlan
- MealPlanDay
- MealSlot

## Grocery Aggregate

- GroceryList
- GroceryItem

## Notification Aggregate

- Notification

## Subscription Aggregate

- Subscription
- Plan

---

# 7. MCD (Modèle Conceptuel de Données)

User 1 ── 1 UserProfile
User 1 ── 1 UserPreferences
User 1 ── N Budget
User 1 ── N PantryItem
User 1 ── N MealPlan
User 1 ── N GroceryList
User 1 ── N Notification
User 1 ── N Subscription

MealPlan 1 ── N MealPlanDay
MealPlanDay 1 ── N MealSlot
MealSlot N ── 1 Recipe

Recipe 1 ── N RecipeIngredient
Recipe 1 ── N RecipeStep

GroceryList 1 ── N GroceryItem

Note: It will be completed if there is anything missing.

---

# 8. UML Class Diagrams (Textual Representation)

```
class User {
  UUID id
  String email
  String passwordHash
  UserStatus status
}

class UserPreferences {
  UUID id
  Integer householdSize
  CookingSkill cookingSkill
  BigDecimal weeklyBudget
}

class PantryItem {
  UUID id
  String ingredientName
  BigDecimal quantity
  Unit unit
  LocalDate expirationDate
}

class Recipe {
  UUID id
  String name
  Integer prepTimeMinutes
  Difficulty difficulty
}

class MealPlan {
  UUID id
  LocalDate weekStartDate
  BigDecimal estimatedCost
}
```

Note: It will be completed if there is anything missing.

# 9. Database Schema (PostgreSQL)

## 9.1 Database Design Principles

The backend of PlatePilot uses PostgreSQL as the primary transactional database.

### Design Principles

- Third Normal Form (3NF) for transactional consistency
- UUID primary keys
- Foreign key constraints
- Audit columns on all business tables
- Soft delete only where business value justifies retention
- Indexes on high-frequency query paths
- Monetary values stored as `NUMERIC(12,2)`
- UTC timestamps using `TIMESTAMP WITH TIME ZONE`

## 9.2 Standard Audit Columns

Every major table includes:

id UUID PRIMARY KEY
created_at TIMESTAMPTZ NOT NULL
updated_at TIMESTAMPTZ NOT NULL
created_by UUID NULL
updated_by UUID NULL
version BIGINT NOT NULL

The `version` column is used by Spring Data JPA for optimistic locking.

## 9.3 Core Tables by Module

### Identity & Access

- `users`
- `roles`
- `permissions`
- `user_roles`
- `role_permissions`
- `refresh_tokens`
- `email_verification_tokens`
- `password_reset_tokens`

### User Profile

- `user_profiles`
- `notification_settings`

### Preferences

- `user_preferences`
- `user_dietary_restrictions`
- `user_allergies`
- `user_goals`
- `user_preferred_cuisines`

### Budget

- `budgets`
- `budget_adjustments`
- `budget_snapshots`

## Pantry

- `pantry_items`

## Recipes

- `recipes`
- `recipe_ingredients`
- `recipe_steps`
- `nutrition_info`

## Meal Planning

- `meal_plans`
- `meal_plan_days`
- `meal_slots`

## Grocery

- `grocery_lists`
- `grocery_items`

## Notifications

- `notifications`

## Subscription

- `plans`
- `subscriptions`

## Analytics

- `analytics_events`

## 9.4 Example Table Definitions

### users

```
CREATE TABLE users (
    id UUID PRIMARY KEY,
    email VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    status VARCHAR(50) NOT NULL,
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,
    last_login_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL,
    version BIGINT NOT NULL
);
```

### user_preferences

```
CREATE TABLE user_preferences (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL UNIQUE REFERENCES users(id),
    household_size INTEGER NOT NULL,
    cooking_skill VARCHAR(50) NOT NULL,
    weekly_budget NUMERIC(12,2) NOT NULL,
    max_cooking_time_minutes INTEGER NOT NULL,
    language_code VARCHAR(10) NOT NULL DEFAULT 'en',
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL,
    version BIGINT NOT NULL
);
```

### pantry_items

```
CREATE TABLE pantry_items (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    ingredient_name VARCHAR(255) NOT NULL,
    quantity NUMERIC(12,3) NOT NULL,
    unit VARCHAR(50) NOT NULL,
    expiration_date DATE,
    category VARCHAR(100),
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL,
```

```
    version BIGINT NOT NULL
);
```

### meal_plans

```
CREATE TABLE meal_plans (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    week_start_date DATE NOT NULL,
    status VARCHAR(50) NOT NULL,
    estimated_total_cost NUMERIC(12,2),
    generated_at TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL,
    version BIGINT NOT NULL,
    UNIQUE(user_id, week_start_date)
);
```

## 9.5 Indexing Strategy

### High-Priority Indexes

- `users(email)`
- `pantry_items(user_id, expiration_date)`
- `meal_plans(user_id, week_start_date)`
- `grocery_lists(user_id, created_at DESC)`
- `notifications(user_id, is_read, created_at DESC)`
- `recipes(difficulty, prep_time_minutes)`

### Full-Text Search

Optional PostgreSQL full-text indexes can be added to recipe names and descriptions.

---

## 9.6 Migration Strategy

All schema changes are versioned using Flyway.

```
src/main/resources/db/migration/
├── V1__initial_schema.sql
├── V2__seed_roles.sql
```

```
├── V3__create_recipe_tables.sql
└── ...
```

# 10. API Design

## 10.1 API Design Principles

The backend exposes a versioned REST API over HTTPS.

### Principles

- JSON request and response bodies
- Versioned URLs (`/api/v1/...`)
- Resource-oriented naming
- Stateless authentication
- Consistent error handling
- Pagination for collection endpoints
- Idempotent updates when appropriate
- OpenAPI documentation

---

## 10.2 Base URL

https://api.platepilot.com/api/v1

---

## 10.3 Authentication Endpoints

| Method | Endpoint | Purpose |
|---|---|---|
| POST | `/auth/register` | Create account |
| POST | `/auth/login` | Authenticate user |
| POST | `/auth/refresh` | Refresh access token |
| POST | `/auth/logout` | Revoke current session |
| POST | `/auth/forgot-password` | Request reset token |

| | | |
|---|---|---|
| POST | `/auth/reset-password` | Reset password |

---

## 10.4 Preferences Endpoints

| Method | Endpoint |
|---|---|
| GET | `/preferences/me` |
| PUT | `/preferences/me` |

---

## 10.5 Budget Endpoints

| Method | Endpoint |
|---|---|
| GET | `/budgets/current` |
| POST | `/budgets` |
| POST | `/budgets/{id}/adjustments` |
| GET | `/budgets/history` |

---

## 10.6 Pantry Endpoints

| Method | Endpoint |
|---|---|
| GET | `/pantry/items` |
| POST | `/pantry/items` |
| PUT | `/pantry/items/{id}` |

| | |
|---|---|
| DELETE | `/pantry/items/{id}` |

---

## 10.7 Meal Planning Endpoints

| Method | Endpoint |
|---|---|
| POST | `/meal-plans/generate` |
| GET | `/meal-plans/current` |
| GET | `/meal-plans/{id}` |
| POST | `/meal-plans/{id}/replace-meal` |
| POST | `/meal-plans/{id}/lock-meal` |

---

## 10.8 Grocery Endpoints

| Method | Endpoint |
|---|---|
| POST | `/grocery-lists/generate` |
| GET | `/grocery-lists/current` |
| PUT | `/grocery-items/{id}` |
| POST | `/grocery-lists/{id}/share` |

---

## 10.9 Quick Meal Endpoint

| Method | Endpoint |
| --- | --- |
| POST | `/recommendations/quick-meal` |

---

## 10.10 Notifications Endpoints

| Method | Endpoint |
| --- | --- |
| GET | `/notifications` |
| PUT | `/notifications/{id}/read` |
| PUT | `/notifications/read-all` |

---

## 10.11 Subscription Endpoints

| Method | Endpoint |
| --- | --- |
| GET | `/subscriptions/current` |
| POST | `/subscriptions/checkout-session` |
| POST | `/subscriptions/webhooks/stripe` |

---

## 10.12 Standard Response Structure

```
{
  "data": {},
  "meta": {
    "timestamp": "2026-05-16T12:00:00Z"
  }
}
```

---

## 10.13 Error Response Structure

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "One or more fields are invalid.",
    "details": [
      {
        "field": "email",
        "message": "must be a valid email address"
      }
    ]
  }
}
```

---

## 10.14 OpenAPI Documentation

All endpoints are documented using OpenAPI and accessible through Swagger UI.

Typical development URL:

http://localhost:8080/swagger-ui.html

---

## 10.15 API Design Conclusion

The API design for PlatePilot is resource-oriented, versioned, secure, and optimized for consumption by the Flutter mobile application.

It provides:

- Clear contracts
- Consistent responses
- Strong validation
- Comprehensive documentation
- Future extensibility

Together with the domain model and PostgreSQL schema, this API design forms the operational contract between the mobile application and the backend platform.

# 11. Authentication & Security

## 11.1 Security Objectives

The backend security architecture is designed to ensure:

- Confidentiality of user data
- Integrity of business operations
- Availability of the API
- Secure authentication
- Fine-grained authorization
- Protection against common web attacks
- Auditability of sensitive actions

The implementation relies on Spring Security, JWT access tokens, refresh tokens, secure password hashing, and HTTPS-only communication.

## 11.2 Authentication Strategy

The recommended approach is:

- Stateless JWT access tokens
- Long-lived refresh tokens persisted in the database
- Token rotation on refresh
- Revocation on logout
- Support for email/password login
- Optional social login using OAuth2

### Token Types

#### Access Token

- Short-lived (e.g., 15 minutes)
- Contains user identity and roles
- Sent in the `Authorization: Bearer <token>` header

#### Refresh Token

- Longer-lived (e.g., 30 days)
- Stored server-side in the `refresh_tokens` table
- Rotated at each refresh request

### 11.3 Password Security

Passwords are never stored in plaintext.

Recommended hashing algorithm:

- BCrypt via Spring Security's `BCryptPasswordEncoder`

Security practices:

- Minimum password policy
- Password reset tokens
- Rate limiting on login attempts

---

### 11.4 Email Verification

After registration:

1. A verification token is generated.
2. The user receives an email containing a secure link.
3. The account is marked as verified after successful confirmation.

Benefits:

- Prevents fake accounts
- Improves account recovery reliability

---

### 11.5 Password Reset Flow

1. User requests password reset.
2. Secure one-time token is created.
3. Email with reset link is sent.
4. Token is validated.
5. Password is updated.
6. Existing sessions may be revoked.

---

### 11.6 Security Filter Chain

Main filters:

- JWT authentication filter
- Exception translation filter
- CORS configuration
- CSRF disabled for stateless APIs
- Security headers

---

### 11.7 Recommended HTTP Security Headers

- `Strict-Transport-Security`
- `X-Content-Type-Options`
- `X-Frame-Options`
- `Referrer-Policy`
- `Content-Security-Policy` (where applicable)

---

### 11.8 Rate Limiting

Recommended for:

- Login
- Registration
- Password reset
- Meal generation endpoints

Possible implementation:

- Redis-backed token bucket algorithm

---

### 11.9 Secrets Management

Sensitive configuration should be stored using:

- AWS Secrets Manager or
- AWS Systems Manager Parameter Store

Never commit secrets to source control.

---

### 11.10 Audit Logging

Audit events include:

- Login success/failure
- Password changes
- Subscription changes
- Administrative actions

---

### 11.11 Security Best Practices Summary

- HTTPS everywhere
- JWT + refresh token rotation
- BCrypt password hashing
- Role-based authorization
- Input validation
- Rate limiting
- Secret management
- Audit logging

---

# 12. Authorization Model

## 12.1 Authorization Strategy

Authorization is based on **Role-Based Access Control (RBAC)**.

Users are assigned one or more roles, and roles are associated with permissions.

---

## 12.2 Core Roles

### USER

Standard application user.

### ADMIN

Internal administrative user.

### SYSTEM

Technical role for scheduled processes and internal automation.

---

## 12.3 Permissions Examples

- `preferences:read`
- `preferences:update`
- `meal-plan:generate`
- `pantry:update`
- `subscription:manage`
- `admin:users:read`

---

## 12.4 Premium Feature Access

Premium access is controlled separately from RBAC through subscription entitlements.

Examples:

- Unlimited meal plans
- Advanced pantry automation

- Shared household accounts

---

### 12.5 Authorization Enforcement

Authorization is enforced:

- At controller level using `@PreAuthorize`
- Within application services when business checks are required

Example:

```
@PreAuthorize("hasRole('USER')")

public MealPlanResponse generateMealPlan(...) { ... }
```

---

# 13. Service Layer Architecture

## 13.1 Purpose

The service layer orchestrates business use cases and acts as the boundary between controllers and the domain model.

---

## 13.2 Application Service Responsibilities

- Validate business preconditions
- Coordinate repositories
- Manage transactions
- Publish domain events
- Return DTOs

---

## 13.3 Use Case-Oriented Design

Each major use case is implemented as a dedicated service or use case class.

Examples:

- `RegisterUserUseCase`
- `CompleteOnboardingUseCase`
- `GenerateMealPlanUseCase`
- `GenerateGroceryListUseCase`
- `AdjustBudgetUseCase`

---

## 13.4 Transaction Management

Transactions are defined using `@Transactional`.

- Read-only queries use `@Transactional(readOnly = true)`
- Commands use standard transactions

---

### 13.5 Service Layer Package Structure

application/

├── usecase/

├── Controller/

├── Repository/

├── service/

├── Model/

├── EXceptionHandler/

├── Configuration/

├── dto/

├── mapper/

└── event/

---

# 14. Repository Layer

## 14.1 Purpose

The repository layer abstracts persistence operations and isolates the domain from database-specific details.

---

## 14.2 Repository Interfaces

Defined in the domain or application layer.

Examples:

- `UserRepository`
- `PantryItemRepository`
- `RecipeRepository`
- `MealPlanRepository`

---

## 14.3 Implementations

Implemented using Spring Data JPA in the infrastructure layer.

---

## 14.4 Custom Queries

Custom methods are used for performance-sensitive operations.

Examples:

- Find expiring pantry items
- Search recipes by constraints
- Retrieve current budget

---

## 14.5 Caching

Selected repository methods may be annotated with Spring Cache backed by Redis.

---

# 15. AI Orchestration Layer

## 15.1 Purpose

Coordinates interactions with external AI services while keeping the core domain deterministic and testable.

---

## 15.2 Responsibilities

- Build prompts
- Send requests to OpenAI
- Parse structured responses
- Validate outputs
- Fallback to rule-based logic if needed

---

## 15.3 Design Principle

AI is used as an enrichment mechanism, not as the primary source of truth for critical business rules.

---

## 15.4 Core Components

- `PromptBuilder`
- `AiClient`
- `ResponseParser`
- `AiGuard`
- `AiOrchestrator`

---

# 16. Recommendation Engine

## 16.1 Purpose

The recommendation engine is the core decision system of PlatePilot.

It generates meal plans that satisfy all user constraints described in our product documentation .

---

## 16.2 Mandatory Inputs

- Household size
- Weekly budget
- Remaining budget
- Cooking skill
- Maximum cooking time
- Dietary restrictions
- Allergies
- Pantry inventory
- Preferred cuisines
- User goals

## 16.3 Processing Pipeline

1. Load candidate recipes.
2. Filter invalid recipes.
3. Estimate cost.
4. Score pantry utilization.
5. Rank candidates.
6. Optimize weekly variety.
7. Persist generated plan.

## 16.4 Scoring Criteria

- Budget fit
- Pantry utilization
- Preparation time fit
- Skill fit
- Goal alignment
- Variety contribution

# 17. Pantry Recognition Pipeline

## 17.1 Purpose

Automates pantry updates based on barcode and OCR scans.

The mobile application uses Google ML Kit for on-device barcode and text recognition .

---

## 17.2 Pipeline Steps

1. Mobile scans product.
2. Barcode or OCR text is extracted.
3. Structured payload is sent to backend.
4. Ingredient normalization is performed.
5. Category and default unit are inferred.
6. Pantry item is created or updated.

---

## 17.3 Backend Responsibilities

- Canonical ingredient mapping
- Duplicate detection
- Quantity normalization
- Confidence-based validation

---

# 18. Budget Optimization Engine

## 18.1 Purpose

Ensures meal plans remain within the user's budget while maximizing value.

---

## 18.2 Inputs

- Weekly budget
- Current spending
- Household size
- Candidate recipe costs

---

## 18.3 Optimization Goals

- Stay within budget
- Maximize pantry usage
- Minimize waste
- Preserve nutritional diversity

---

## 18.4 Outputs

- Cost-compliant meal selection
- Estimated savings
- Budget utilization metrics

---

# 19. Notification System

## 19.1 Purpose

Delivers timely and relevant reminders and alerts.

---

## 19.2 Notification Types

- Pantry expiration alerts
- Budget threshold alerts
- Weekly planning reminders
- Grocery reminders
- Premium prompts

These categories are defined in the MVP Refinement Blueprint .

---

## 19.3 Delivery Channels

- In-app notifications
- Push notifications via Firebase Cloud Messaging
- Email (optional)

---

## 19.4 Scheduling

Scheduled jobs periodically:

- Detect expiring items
- Evaluate budget thresholds
- Trigger weekly reminders

---

# 20. Subscription & Billing

## 20.1 Purpose

Manages premium access and monetization.

## 20.2 Recommended Provider

Stripe for payment processing and subscription lifecycle management.

## 20.3 Core Entities

- Plan
- Subscription
- FeatureEntitlement
- BillingEvent

## 20.4 Subscription States

- Trialing
- Active
- Past Due
- Canceled
- Expired

## 20.5 Premium Features

- Unlimited meal plans
- Advanced pantry automation
- Budget optimization insights
- Shared household accounts

## 20.6 Webhook Processing

The backend receives signed webhook events from Stripe to synchronize:

- Subscription creation
- Renewal
- Payment failure
- Cancellation

## 20.7 Access Control Integration

Application services check feature entitlements before enabling premium functionality.

# 21. Event-Driven Architecture

## 21.1 Purpose

Although the system is implemented as a Modular Monolith, many business processes are naturally asynchronous and should be decoupled through domain and application events.

Event-driven design enables:

- Loose coupling between modules
- Better extensibility
- Improved maintainability
- Asynchronous processing
- Auditability

## 21.2 Event Categories

### Domain Events

Represent meaningful business state changes.

Examples:

- `UserRegisteredEvent`
- `OnboardingCompletedEvent`
- `PantryItemAddedEvent`
- `MealPlanGeneratedEvent`
- `BudgetAdjustedEvent`
- `SubscriptionActivatedEvent`

### Application Events

Used to trigger side effects.

Examples:

- Send verification email
- Recalculate recommendations
- Push notifications
- Record analytics

## 21.3 Event Flow Example

MealPlanGeneratedEvent

├── GenerateGroceryListHandler

├── RecordAnalyticsHandler

├── SendNotificationHandler

└── CacheWarmupHandler

## 21.4 Spring Implementation

Recommended mechanisms:

- `ApplicationEventPublisher`
- `@EventListener`
- `@TransactionalEventListener`

Use `AFTER_COMMIT` to ensure handlers execute only after successful transactions.

## 21.5 Future Evolution

If asynchronous workloads increase, events can later be externalized to Apache Kafka or RabbitMQ without changing the domain model.

# 22. Caching Strategy

## 22.1 Objectives

Caching reduces:

- API latency
- Database load
- Repeated AI calls
- Computational overhead

The primary cache technology is Redis.

## 22.2 Cache Candidates

### Frequently Read Data

- User preferences
- Active subscription entitlements
- Recipe metadata
- Ingredient normalization mappings

### Computed Results

- Weekly meal plans
- Grocery lists
- Recommendation candidate sets

### Security Data

- Rate limiting counters
- Session metadata

## 22.3 Cache Keys

preferences:{userId}

budget:{userId}

pantry:{userId}

mealplan:{userId}:{weekStart}

grocery:{userId}:{mealPlanId}

entitlements:{userId}

---

## 22.4 Expiration Policy

| Data Type | TTL |

| --- | --- |

| Preferences | 1 hour |

| Pantry | 15 minutes |

| Meal Plans | 24 hours |

| Recipe Metadata | 12 hours |

| Rate Limits | Variable |

## 22.5 Cache Invalidation

Whenever source data changes:

- Preferences update → invalidate preferences and recommendation caches
- Pantry update → invalidate pantry and recommendation caches
- Budget update → invalidate budget and recommendation caches

# 23. File Storage

## 23.1 Purpose

Persistent file storage is required for:

- User avatars
- Pantry scan images (optional)
- Recipe photos
- Exported grocery lists
- Generated reports

## 23.2 Recommended Storage

Amazon S3 is the authoritative object storage service or Cloudinary.

## 23.3 Bucket Structure

platepilot-prod/

├── avatars/

├── pantry-scans/

├── recipe-images/

├── exports/

└── temp/

## 23.4 Upload Strategy

Recommended approach:

1. Backend generates a pre-signed upload URL.
2. Mobile app uploads directly to S3 or Cloudinary.
3. Backend stores metadata only.

Benefits:

- Reduced backend bandwidth
- Better scalability, Lower latency

# 24. Monitoring & Observability

## 24.1 Objectives

The platform must provide visibility into:

- Performance
- Errors
- Infrastructure health
- Business KPIs

## 24.2 Three Pillars

### Metrics

Collected using Micrometer and Prometheus.

### Logs

Structured JSON logs.

### Traces

Distributed tracing using OpenTelemetry.

## 24.3 Dashboards

Recommended visualization with Grafana.

Key dashboards:

- API performance
- Database health
- Recommendation latency
- AI usage and cost
- Business KPIs

---

## 24.4 Error Tracking

Use Sentry for:

- Exception aggregation
- Stack traces
- Release correlation

# 25. Testing Strategy

## 25.1 Testing Pyramid

End-to-End Tests

Integration Tests

Unit Tests (largest layer)

## 25.2 Unit Tests

Frameworks:

- JUnit 5
- Mockito
- AssertJ

Targets:

- Domain entities
- Value objects
- Use cases
- Recommendation scoring

## 25.3 Integration Tests

Use:

- Testcontainers
- PostgreSQL containers
- Redis containers

## 25.4 API Tests

Use Spring Boot Test and `MockMvc`.

## 25.5 End-to-End Tests

Validate complete workflows:

- Registration
- Onboarding
- Meal plan generation
- Grocery generation
- Subscription activation

## 25.6 Coverage Targets

- Domain layer: ≥ 90%
- Application layer: ≥ 85%
- Overall backend: ≥ 80%

# 26. CI/CD

## 26.1 Objectives

Automate:

- Build
- Testing
- Security checks
- Docker image creation
- Deployment

## 26.2 Recommended Platform

GitHub GitHub Actions.

## 26.3 Pipeline Stages

1. Checkout
2. Static analysis
3. Unit tests
4. Integration tests
5. Build JAR
6. Build Docker image
7. Push to Amazon Elastic Container Registry
8. Deploy to AWS
9. Smoke tests

# 27. Infrastructure on Amazon Web Services

## 27.1 MVP Architecture

Flutter App

↓ HTTPS

AWS App Runner/E2

↓

Amazon RDS PostgreSQL/SuperBase

↓

Amazon ElastiCache Redis/Spring Redis

↓

Amazon S3

↓

CloudWatch

## 27.2 Core Services

- AWS App Runner
- Amazon RDS for PostgreSQL
- Amazon ElastiCache for Redis
- Amazon S3
- Amazon CloudWatch
- AWS Secrets Manager
- Amazon Elastic Container Registry

## 27.3 Growth Architecture

When scale increases, migrate to:

- Amazon ECS or Amazon EKS
- Multi-AZ databases
- CDN and additional observability tooling

# 28. Performance & Scalability

## 28.1 Performance Targets

| Metric | Target |
| --- | --- |
| Standard API latency (P95) | < 300 ms |
| Cached API latency (P95) | < 100 ms |
| Meal plan generation | < 60 seconds |
| Quick meal suggestion | < 5 seconds |
| Availability | 99.9% |

## 28.2 Scalability Strategy

- Horizontal scaling of application containers
- Read-heavy caching
- Direct-to-S3 uploads
- Asynchronous event processing
- Database indexing and optimization

# 29. Data Privacy & Compliance

## 29.1 Privacy Principles

The platform should adhere to privacy-by-design principles:

- Data minimization
- Purpose limitation
- Security by default
- Transparency

## 29.2 User Rights

Users should be able to:

- Access their data
- Export their data
- Delete their account
- Manage notification and privacy settings

## 29.3 Sensitive Data

Potentially sensitive information includes:

- Dietary restrictions
- Allergies
- Household composition

These data require strong protection but are not treated as payment data; payment processing is delegated to Stripe.

# 30. Development Roadmap

## Phase 1 — Foundation

- Project setup
- Modular structure
- Security baseline
- Database migrations

## Phase 2 — Core Domains

- Authentication
- Preferences
- Budget
- Pantry
- Recipes

## Phase 3 — Recommendation Engine

- Meal planning
- Grocery generation
- Quick meal mode

## Phase 4 — AI & Automation

- AI orchestration
- Pantry recognition
- Budget optimization

## Phase 5 — Product Services

- Notifications
- Analytics
- Subscription & Billing

## Phase 6 — Production Readiness

- Observability
- Performance tuning

- Security hardening
- Compliance

### Phase 7 — Beta Launch

- Backend deployment
- Frontend integration
- Internal QA
- Private beta

---

# Final Strategic Conclusion

With sections 1 through 30 now defined, PlatePilot has a complete, production-grade backend architecture blueprint.

This blueprint establishes:

- Clear domain boundaries
- Robust data modeling
- Secure authentication and authorization
- Deterministic recommendation and optimization engines
- AI orchestration
- Cloud-native infrastructure
- Monitoring and automated delivery
- Privacy and compliance foundations
- A phased implementation roadmap

Combined with our Product Bible, PRD, UX/UI Blueprint, and Flutter frontend, PlatePilot now possesses a rigorous technical and strategic foundation to evolve from concept to a scalable global SaaS platform.